

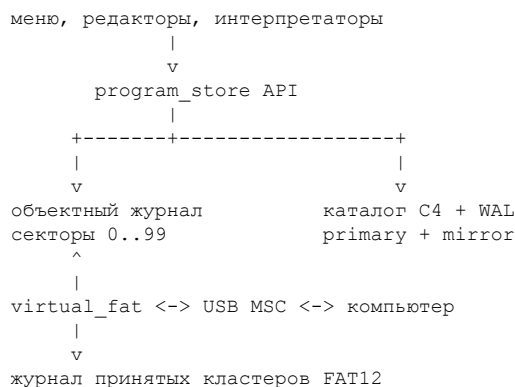
Внутреннее файловое хранилище MK61s mini

Версия документа: 10.07.2026

MK61s mini хранит программы, тексты, состояния и шрифты во внешней SPI flash. Это не FAT-раздел на микросхеме. Постоянные файлы находятся в собственном журналируемом объектном хранилище `program_store`, а USB Mass Storage показывает поверх него синтезируемый на лету FAT12-диск `virtual_fat`.

Документ описывает оба слоя, физический формат записей, LZSS-сжатие, каталог C4 и WAL, сборку мусора, промежуточный журнал USB-записей и правила импорта. Эталонная реализация находится в `code/program_store.cpp`, `code/program_store.hpp` и `code/virtual_fat.cpp`.

Архитектура



`program_store` является источником истины. FAT12 не хранится во flash как готовый образ: boot sector, обе FAT и корневой каталог формируются при чтении. При записи с компьютера 512-байтные кластеры сначала попадают в отдельный flash-журнал, а после получения согласованной записи каталога собираются в объект `program_store`.

Публичные ограничения

Параметр	Значение
Размер erase-сектора SPI flash	4096 байт
Секторов объектного журнала	100
Максимум объектов всех типов	128
Длина внутреннего имени	1..15 байт
Логический размер объекта	1..1536 байт
Размер сектора/кластера FAT12	512 байт
Кластеров виртуального диска	800
Одновременных незавершённых USB-записей	32

Квота 128 общая: это не 128 файлов каждого типа. Пустые объекты постоянное хранилище не создаёт. Нулевые `directory entries`, которыми операционная система начинает копирование, считаются временными `placeholders` и удаляются при `sync`.

Лимит 1536 байт равен трём кластерам FAT12. Сжатие уменьшает физический расход flash, но не увеличивает публичный логический лимит файла.

Типы объектов

ProgramType	ID в C4	Тег записи	USB-расширение	Назначение
MK61	0	M1	.m61	Сценарий/программа МК-61
FOCAL	2	F1	.foc	Исходник FOCAL
TINYBASIC	3	B2	.tbi	Исходник TinyBASIC
TEXT	4	T1	.txt	Текст
MK61_STATE	5	M2	.state.txt	Снимок состояния
FONT	6	F3	.fmk	Шрифт FMK1

В физической записи старший бит второго байта тега означает внешнее LZSS-сжатие. Например, сжатый FOCAL имеет байты 'F' и '1' | 0x80. В C4 тот же признак хранится как старший бит числового ID типа.

Для FONT внешний признак сжатия запрещён. FMK1 использует собственный битовый RLE, описанный отдельно в MK61s-mini-FMK.md.

Карта SPI flash

Номера ниже заданы в 4096-байтных erase-секторах.

Сектор	Адрес	Назначение
0..99	0x00000..0x63FFF	Журнал объектов program_store
100	0x64000	Основная копия каталога
101	0x65000	Журнал настроек устройства
102..229	0x66000..0xE5FFF	Журнал staging для USB FAT12
230	0xE6000	Зеркальная копия каталога

Диапазон staging ограничен 128 секторами, но фактически используется только часть, которая существует на установленной flash. Зеркальный каталог также используется только если сектор 230 доступен. На штатной W25Q128 весь диапазон доступен.

Подсистемы после сектора 230 не входят в описываемый формат. Сектор настроек 101 зарезервирован между каталогом и staging и не является файловым объектом.

Сектор объектного журнала

Каждый из секторов 0..99 может быть пустым, активным или foreign. Активный сектор начинается с семибайтного заголовка:

Смещение	Размер	Поле	Значение
0	1	tag0	ASCII S
1	1	tag1	ASCII 1
2	1	state	0xFF при записи, затем 0x7F
3	4	generation	Ненулевое поколение, little-endian

После заголовка с offset 7 последовательно расположены записи переменной длины. Отдельного выравнивания между ними нет. Новая запись всегда дописывается в стертые байты 0xFF; существующий payload никогда не переписывается на месте.

Состояния выбраны так, чтобы переходы требовали только сброса битов NOR flash:

```
0xFF STATE_WRITING
0x7F STATE_ACTIVE
0x3F STATE_DELETED
```

Заголовок сектора коммитится в том же порядке: сначала тег и поколение при STATE_WRITING, последней операцией состояние меняется на STATE_ACTIVE.

foreign - не физический тег, а состояние в каталоге. Так помечается сектор, который нельзя считать частью актуального журнала: например, старая копия после перемещения данных или сектор при загрузке без валидного каталога. Его можно безопасно стереть при последующем выделении места.

Физическая запись объекта

```
+0 8 байт header
+8 name_len байт имени, без завершающего NUL
stored_len байт payload
```

Заголовок:

Смещение	Размер	Поле	Описание
0	1	tag0	Первый байт тега типа
1	1	tag1_flags	Второй байт тега; бит 7 означает LZSS
2	1	state	0xFF, 0x7F или 0x3F
3	1	name_len	Длина имени 1..15
4	2	data_len	Логическая несжатая длина, little-endian
6	2	crc16	CRC заголовочных полей, имени и сохранённого payload

Полная физическая длина:

```
total_len = 8 + name_len + stored_len
```

Для сырой записи `stored_len == data_len`. Для сжатой записи физическая длина не хранится отдельным числом: парсер декодирует LZSS до `data_len`, определяет число потреблённых байтов и требует $0 < \text{stored_len} < \text{data_len}$.

Одна запись не пересекает границу 4096-байтного сектора. Благодаря публичному лимиту максимальная сырая запись занимает $8 + 15 + 1536 = 1559$ байт и всегда помещается после семибайтного заголовка сектора.

Протокол добавления

- 1 Выбирается активный сектор с достаточным стертым хвостом.
- 2 Записываются тег, STATE_WRITING, длины и CRC.
- 3 Записываются имя и сырой либо сжатый payload.
- 4 Одной записью байт состояния меняется на STATE_ACTIVE.
- 5 Обновляется RAM-индекс и дописывается WAL каталога.
- 6 При замене старой версии её состояние по возможности меняется на STATE_DELETED; место остаётся занятым до GC.

Если запись payload или финального состояния не удалась, хвост сектора запечатывается: used считается равным 4096. Следующая операция не пытается писать поверх частично запрограммированных битов.

Повторная запись тех же байтов под тем же типом и именем является no-op: payload сравнивается с актуальной записью, flash и каталог не изменяются.

CRC записи

Используется CRC-16/CCITT-FALSE с начальным значением 0xFFFF и полиномом 0x1021. CRC вычисляется в таком порядке:

```
tag0
tag1_flags
name_len
low(data_len)
high(data_len)
name[0..name_len-1]
stored_payload[0..stored_len-1]
```

Поле состояния, само поле CRC, поколение и свободный хвост сектора не входят в контрольную сумму. Для сжатого объекта CRC защищает именно LZSS-поток, а не развёрнутый результат.

Внешнее LZSS-сжатие объектов

LZSS-поток состоит из битов MSB-first и завершается нулевыми битами до границы байта. Формат не содержит отдельного заголовка или словаря.

Литерал

```
1 dddddddd
```

После бита 1 идут 8 бит исходного байта. Стоимость - 9 бит.

Ссылка

```
0 dddddddd 111
```

Поле	Биты	Декодированное значение
distance_code	9	distance = code + 1, диапазон 1..512
length_code	3	length = code + 1, допустимо 2..8

Код длины 0 формально читается как длина 1, но декодер отвергает такую ссылку: совпадение должно содержать хотя бы 2 байта. Расстояние не может быть больше уже выданного объёма, а ссылка не может вывести результат за data_len.

Копирование допускает перекрытие источника и результата. Поэтому короткая история может эффективно разворачивать длинную серию повторов.

Стратегия кодировщика

В каждой позиции кодировщик ищет самое длинное совпадение длиной до 8 во всех расстояниях от 1 до $\min(\text{position}, 512)$. При одинаковой длине сохраняется меньшее расстояние, найденное первым. Совпадение длиной 2 и более кодируется ссылкой, иначе литералом.

Сжатие выполняется в два прохода:

- 1 Поток только измеряется и одновременно рассчитывается его CRC.
- 2 Если число полученных байтов строго меньше data_len, поток повторно кодируется прямо во flash.

Если выигрыша нет, сохраняется исходный payload. Поэтому флаг сжатия всегда означает реальное уменьшение физической длины.

Политика применения

- Внутренний program_store::write пробует LZSS для всех типов, кроме FONT.
- program_store::write_from_usb сохраняет допустимый USB-импорт без LZSS, чтобы время sync было предсказуемым.
- FONT всегда хранится внешне без сжатия и использует только внутренний RLE формата FMK1.
- GC копирует сжатую запись как есть, не разворачивая и не пережимая её.

read прозрачно разворачивает весь объект. read_range для сжатой записи также начинает декодирование с начала, но держит только кольцевую историю 512 байт и выдаёт запрошенный диапазон. Произвольного seek внутри LZSS-потока нет.

Проверка конца потока

Декодирование заканчивается после получения ровно data_len байтов. Остаток уже загруженного последнего байта должен быть нулевым. Число фактически потреблённых байтов обязано совпасть с stored_len, восстановленным при разборе записи.

RAM-индекс

При работе каталог загружается в массив максимум из 128 IndexEntry. На объект хранятся:

- внутреннее имя в 16-байтном буфере;
- offset и номер сектора;
- логическая и полная физическая длина;
- числовой тип с битом сжатия.

Текущая структура занимает 24 байта на объект. Ключ объекта - пара (ProgramType, name): одинаковые имена разных типов допустимы.

Каталог C4

Каталог ускоряет загрузку и является точкой публикации изменений. Текущий формат C4 использует компактные ссылки на записи: имя не дублируется в checkpoint, а читается из заголовка соответствующего объекта.

Две копии

Основная копия находится в секторе 100, зеркальная - в секторе 230. При полном checkpoint прошивка стирает неактивную копию, записывает её с новым поколением и последним меняет состояние на ACTIVE. Старая валидная копия не стирается.

При старте обе копии проверяются, выбирается валидная с более новым 32-битным поколением. Сравнение поколений учитывает циклическое переполнение. Если полная загрузка выбранной копии не удалась, проверяется вторая.

Если зеркальный сектор физически отсутствует, используется только сектор 100, что уменьшает устойчивость к отказу именно во время checkpoint.

Заголовок C4

Смещение	Размер	Поле	Описание
0	1	tag0	ASCII c
1	1	tag1	ASCII 4
2	1	state	0xFF при записи, 0x7F после commit
3	1	sector_count	Должно быть 100
4	1	entry_count	0..128
5	2	snapshot_len	Длина checkpoint, little-endian
7	2	crc16	CRC checkpoint
9	4	generation	Поколение копии, little-endian

Длина checkpoint:

```
snapshot_len = 13 + 100 * 9 + entry_count * 9
```

CRC использует тот же CCITT-FALSE. Он включает теги, поля 3..6, поколение и всё тело checkpoint, но исключает состояние и байты CRC.

Состояние сектора в C4

После заголовка идут ровно 100 девятибайтных записей:

Смещение	Размер	Поле
0	1	Ровно один флаг: EMPTY=1, ACTIVE=2 или FOREIGN=4
1	4	Поколение сектора
5	2	used, занятый физический префикс
7	2	live, сумма длин актуальных объектов

Для активного сектора $dead = used - 7 - live$ вычисляется при загрузке. Для пустого сектора поколение и длины нулевые. Для foreign значение used равно 4096, live равно 0.

Элемент C4

Смещение	Размер	Поле
0	1	ID типа и бит 0x80 внешнего сжатия
1	1	Длина имени
2	1	Номер сектора 0..99
3	2	Offset записи в секторе
5	2	Логическая длина data_len
7	2	Полная физическая длина total_len

При загрузке C4 прошивка читает по ссылке восьмибайтный header объекта и имя. Состояние checkpoint-записи может быть как ACTIVE, так и DELETED: более новый WAL upsert уже мог пометить старую запись удалённой, но её идентификационные байты остаются пригодными до следующего checkpoint.

WAL каталога

После checkpoint следующий offset округляется вверх до границы 64 байт. Там последовательно дописываются записи D4 длиной 64 байта. WAL позволяет обычным созданиям, заменам и удалениям не стирать целый сектор каталога.

Смещение	Размер	Поле
0	1	ASCII D
1	1	ASCII 4
2	1	Состояние 0xFF -> 0x7F
3	1	Размер записи, всегда 64
4	2	Последовательность, начинается с 1
6	1	Число обновлений секторов, 0..2
7	1	Операция: 1 upsert, 2 delete

Смещение	Размер	Поле
8	2	CRC WAL-записи
10	20	До двух десятибайтных состояний секторов
30	9	Компактный C4 entry для upsert; для delete нужен тип
39	16	Имя с NUL и нулевым дополнением
55	9	Зарезервированный хвост 0xFF

Десятибайтное состояние сектора состоит из его номера и обычной девятибайтной записи C4. CRC охватывает теги и байты 3..63, кроме собственных байтов 8..9 и состояния 2.

При replay последовательность должна быть непрерывной, состояние активным, CRC правильным, а все ссылки структурно допустимыми. На первой повреждённой или частичной записи replay останавливается и сохраняет состояние после последней валидной записи. Следующая мутация делает полный checkpoint, а не пишет поверх сомнительного хвоста.

Когда новая 64-байтная запись не помещается, создаётся checkpoint в другой копии каталога. Для 128 объектов максимальный C4 checkpoint занимает 2065 байт, поэтому в секторе остаётся место как минимум для нескольких WAL-записей.

Совместимость каталогов

Загрузчик понимает четыре поколения:

Тег	Заголовок	Entry	WAL	Поведение
C1	9 байт	25 байт	нет	Старый checkpoint без поколения
C2	13 байт	25 байт	нет	Checkpoint с поколением
C3	13 байт	25 байт	D3	Старый WAL
C4	13 байт	9 байт	D4	Текущий формат

25-байтный legacy entry содержит те же первые 9 байт и 16 байт имени. После загрузки старого каталога первая мутация создаёт новый атомарный C4 checkpoint. Обратная запись в старом формате не выполняется.

Поведение при старте и сбоях

Последовательность `program_store::init`:

- 1 Сканируются компактные headers журнала staging и восстанавливаются последние ссылки на принятые USB-кластеры.
- 2 Проверяются primary и mirror каталоги.
- 3 Загружается новейший валидный checkpoint.
- 4 Воспроизводится непрерывный валидный префикс WAL.
- 5 Проверяются ссылки, размеры, состояния секторов и идентичность объектов.
- 6 Если после опубликованного used обнаружены запрограммированные байты, хвост сектора карантинизируется и больше не дописывается.

Такой хвост возникает, если объект успел физически записаться, но его WAL не успел закоммититься. Каталог остаётся источником истины, поэтому объект не появляется после перезапуска, а повторная запись идёт в чистое место.

Если ни один каталог невалиден, прошивка не пытается эвристически собрать файлы из сырых записей. Все 100 секторов помечаются `foreign`; последующие записи могут постепенно стирать их. Это защищает от публикации случайных или устаревших данных, но означает, что каталог обязателен для штатного восстановления.

Форматирование стирает 100 секторов данных, основную и доступную зеркальную копию каталога, сбрасывает индекс и логически инвалидирует актуальные `staging records`. После этого записывается пустой `C4 checkpoint`.

Сборка мусора

Удаление и замена не освобождают байты немедленно. GC работает целыми `erase`-секторами:

- 1 Требуется пустой сектор назначения.
- 2 Среди активных выбирается сектор с минимальным объемом `live`-данных, который после уплотнения оставит достаточно места для требуемой записи.
- 3 Актуальные записи копируются побайтно в новый активный сектор с протоколом `WRITING -> ACTIVE`; сжатые `payload` не меняются.
- 4 Каталог публикует новые ссылки и помечает старый сектор `foreign`.
- 5 Только после `commit` каталога старый сектор стирается.
- 6 Вторым `commit` помечает его пустым.

Если питание исчезнет до публикации, старый каталог всё ещё ссылается на старые данные. Если после публикации, новый каталог уже ссылается на копии, а старый сектор остаётся безопасным `foreign`. Цена некоторых сбоев - временная потеря одного свободного сектора, а не потеря опубликованного объекта.

При обычном дописывании система старается иметь два пустых сектора. Один нужен для открытия нового журнала, второй оставляет пространство для GC.

Виртуальный FAT12-диск

USB-хост видит стандартный суперфлоппи FAT12 без таблицы разделов.

Параметр BPB	Значение
Bytes per sector	512
Sectors per cluster	1
Reserved sectors	1
FAT copies	2
Root entries	1024
Media descriptor	0xF8
Data clusters	800
Volume label	MK61S FS
Filesystem label	FAT12

Для 802 FAT entries требуется 1203 байта, то есть 3 сектора на одну FAT. Корневой каталог занимает 64 сектора. При обычной сборке карта LBA такова:

LBA	Назначение
0	Boot sector
1..3	FAT #1
4..6	FAT #2
7..70	Корневой каталог
71..870	800 кластеров данных

Полный логический размер диска - 871 сектор, или 445952 байта. Свободное место FAT12 является интерфейсной ёмкостью для алгоритма копирования и не равно свободному месту объектного журнала. Реальные ограничения всё равно составляют 128 объектов и 1536 байт на объект.

Boot sector, FAT и каталог генерируются из RAM-индекса. Файлы, уже показанные хосту, получают стабильные цепочки кластеров на время USB-сессии. Поэтому удаление соседнего файла не сдвигает данные под смонтированной файловой системой.

Поддерживаются и фрагментированные цепочки, записанные хостом. Прошивка хранит последние FAT-секторы от компьютера, сверяет обе копии FAT по CRC и замораживает цепочку в момент принятия directory entry. Циклы, выход за диапазон и конфликт двух копий FAT отклоняются.

Подкаталоги, hidden/system entries и неподдерживаемые расширения не превращаются во внутренние объекты. Их диапазоны кластеров временно помечаются игнорируемыми, чтобы служебные файлы ОС не засоряли staging.

Имена на FAT12

Внутреннее имя содержит максимум 15 байт. Для простого имени из 1..8 заглавных ASCII-букв, цифр и _ может использоваться обычная запись 8.3. Дефис требует LFN и в коротком alias превращается в _.

Для TEXT и MK61_STATE, а также для длинных или несовместимых имён создаётся LFN. Видимые имена имеют расширения .txt, .state.txt и .fmk. Короткий alias формируется как:

ABCDE~XY.EXT

Первые пять ASCII-букв или цифр берутся из имени, дефис и _ превращаются в _, остальные символы пропускаются, а пустые позиции заполняются _. XY - устойчивый в пределах сессии двухзначный base36-суффикс. Имя, которое уже выглядит как сгенерированный alias, всё равно получает новый alias, чтобы исключить коллизию.

Во внутреннем списке и на USB скрыты объекты с именем, начинающимся с . или _, а также объекты нулевой длины.

Импорт LFN

Распознавание расширения регистронезависимо. .state.txt проверяется раньше .txt. Обычные трёхбуквенные расширения должны быть ASCII.

Базовая часть имени нормализуется так:

- 1 Имя разбивается на слова по пробелам и распространённой пунктуации.
- 2 ASCII-буквы и цифры сохраняются.
- 3 Кириллица транслитерируется, например Ж -> Zh, Ц -> Ts, Щ -> Sch, Ю -> Yu, Я -> Ya; мягкий и твёрдый знаки удаляются.
- 4 Неподдерживаемые символы пропускаются.
- 5 Если слов нет, используется FILE.
- 6 Пока суммарная длина превышает 15, по одному символу отрезается от самого длинного слова.
- 7 Пробелы между словами сохраняются только если исходное соединённое имя уже помещается вместе с ними.

Если нормализованное имя конфликтует с существующим объектом того же типа, добавляется суффикс 1..999 с обрезанием основы. Исключение: если LFN base уже в точности равен допустимому внутреннему имени, копирование считается обновлением этого объекта, а не созданием дубликата.

Короткое имя 8.3 без LFN импортируется в верхнем регистре. В середине base не допускаются пробелы, разрешены только буквы, цифры, _ и -.

Журнал staging для USB

Операционные системы могут записать данные, FAT и directory entries в любом порядке. Нельзя держать до трёх кластеров каждого незавершённого файла только в RAM и нельзя публиковать объект до получения его имени и длины. Поэтому секторы 102..229 образуют отдельный log-structured журнал 512-байтных кластеров.

Разметка одного staging-сектора

Первые 512 байт зарезервированы под компактные headers. Затем находятся семь payload по 512 байт:

```
offset 0      header #0, 8 байт
offset 8      header #1
...
offset 48     header #6
offset 56..511 не используется
offset 512    payload #0
offset 1024   payload #1
...
offset 3584   payload #6
```

Один 4096-байтный сектор хранит 7 records, весь диапазон - максимум 896 последовательных записей. Логическое пространство FAT12 содержит 800 кластеров с номерами 2..801.

Header staging-record

Смещение	Размер	Поле	Описание
0	1	tag0	ASCII w
1	1	state	0xFF при записи, S после commit, C после удаления
2	2	cluster	Номер кластера, little-endian
4	2	crc16	CRC кластера, поколения и 512 байт payload
6	2	generation	Ненулевое 16-битное поколение

Commit выполняется в порядке header с 0xFF, payload, затем единственный байт S. Старая версия того же кластера после публикации новой по возможности меняется с S на C.

CRC считается по little-endian байтам cluster, little-endian байтам generation и всему 512-байтному payload. При чтении повреждённая запись помечается C и удаляется из RAM-индекса.

Восстановление и очистка

При старте сканируются только 8-байтные headers. Для каждого из 800 кластеров выбирается новейшая committed-запись с учётом циклического сравнения 16-битных поколений. Это ограничивает первичное чтение примерно 7 КиБ даже при полном журнале; payload и CRC проверяются лениво при первом чтении.

Один erase-сектор удерживается как резерв. Если сектор больше не содержит актуальных ссылок, он стирается целиком. Если все обычные секторы содержат live records, выбирается самый разреженный, его live-кластеры копируются в резерв, после чего бывший сектор становится новым резервом.

Частичная запись запечатывает только текущий хвост. Ранее committed records того же erase-сектора остаются доступными и могут быть перенесены cleaner-ом.

Транзакция USB-импорта

Data sectors попадают в staging, а прошивка запоминает FAT, включая фрагментированные цепочки. Short entry или проверенная LFN-цепочка задаёт тип, имя, кластер и длину; цепочка проверяется, замораживается и не должна пересекаться с чужой. Полный файл собирается в scratch-буфере, write_from_usb коммитит сырой объект, затем staging-ссылки забываются.

Если объект уже существует и хост изменил только часть его кластеров, сначала читается старая версия, затем поверх неё накладываются принятые кластеры. Так поддерживается частичная перезапись без требования, чтобы хост повторно прислал неизменённые сектора.

При ошибке постоянной записи транзакция и staged payload сохраняются. SYNC возвращает ошибку, но уже принятые данные не отбрасываются. Полный объект публикуется только после успешного program_store commit.

Удаления также откладываются до flush_pending. Сначала directory tombstone проверяется по размеру, кластеру и хвосту short name, затем удаления выполняются от больших индексов к меньшим, поскольку удаление уплотняет RAM-индекс.

Переименование, которое хост представил как delete old + create new, может быть распознано по прежней цепочке и длине. Внутреннее rename реализовано как чтение, запись объекта под новым именем и удаление старого.

Важные свойства и границы

- Постоянный файл существует только после commit записи объекта и WAL/каталога.
- FAT12 является проекцией, а не пригодным для побайтового резервного копирования образом внутренней flash.
- Свободное место FAT12 не показывает реальную фрагментацию или квоту 128 объектов.
- Сжатие прозрачно для API и USB, но USB-импорт сохраняется сырым.
- Повреждённый новый WAL откатывает только неполный хвост, а не предыдущие валидные операции.
- Без валидного каталога автоматического полного scan/recovery объектов нет.
- CRC обнаруживает случайное повреждение, но не является криптографической защитой.
- Имена являются ключами внутри типа; регистр внутреннего имени значим для прямых API-вызовов, хотя расширения USB распознаются без учёта регистра.
- Логическая длина 1536 не увеличивается от хорошего коэффициента LZSS.

Формулы для парсера

```
SECTOR_SIZE = 4096
SECTOR_HEADER_SIZE = 7
RECORD_HEADER_SIZE = 8

raw_total = 8 + name_len + data_len
compressed_total = 8 + name_len + consumed_lzss_bytes

c4_snapshot = 13 + 100 * 9 + entry_count * 9
c4_wal_start = ceil(c4_snapshot / 64) * 64

fat12_bytes = ceil((800 + 2) * 12 / 8) = 1203
fat_sectors = ceil(1203 / 512) = 3
root_sectors = 1024 * 32 / 512 = 64
disk_sectors = 1 + 2 * 3 + 64 + 800 = 871
```

Реализация независимого парсера должна проверять переполнение каждого сложения, границы сектора, допустимость типа и состояния, CRC, монотонность WAL и соответствие логической и физической длин. Нельзя считать сигнатуру достаточной проверкой валидности.